Software Process Management

Lecture 7

Engineering Process Models

- Typically cover only a relatively small fraction of the complete lifecycle of a software product.
- They often elaborate more details than lifecycle models.

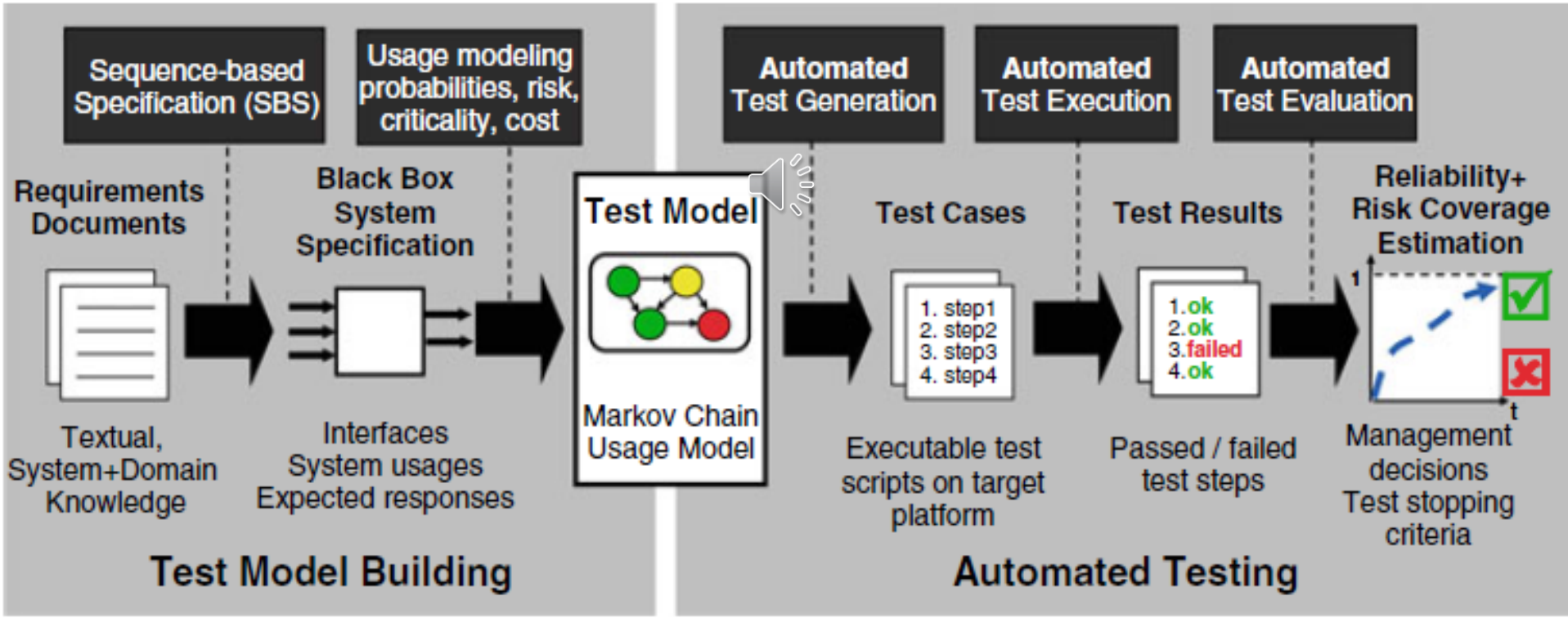


- Still follow the same principles as lifecycle process models.
- Let’s consider some examples

Model Based Statistical Testing

- Model Based Statistical Testing is a usage model based on Marcov chain usage model.
- The usage model is used to derive realistic test cases considering the operational profile of the test object.
- This enables the estimation of system reliability based on the test results.
- Model-based statistical testing consists of several steps, which are shown in:

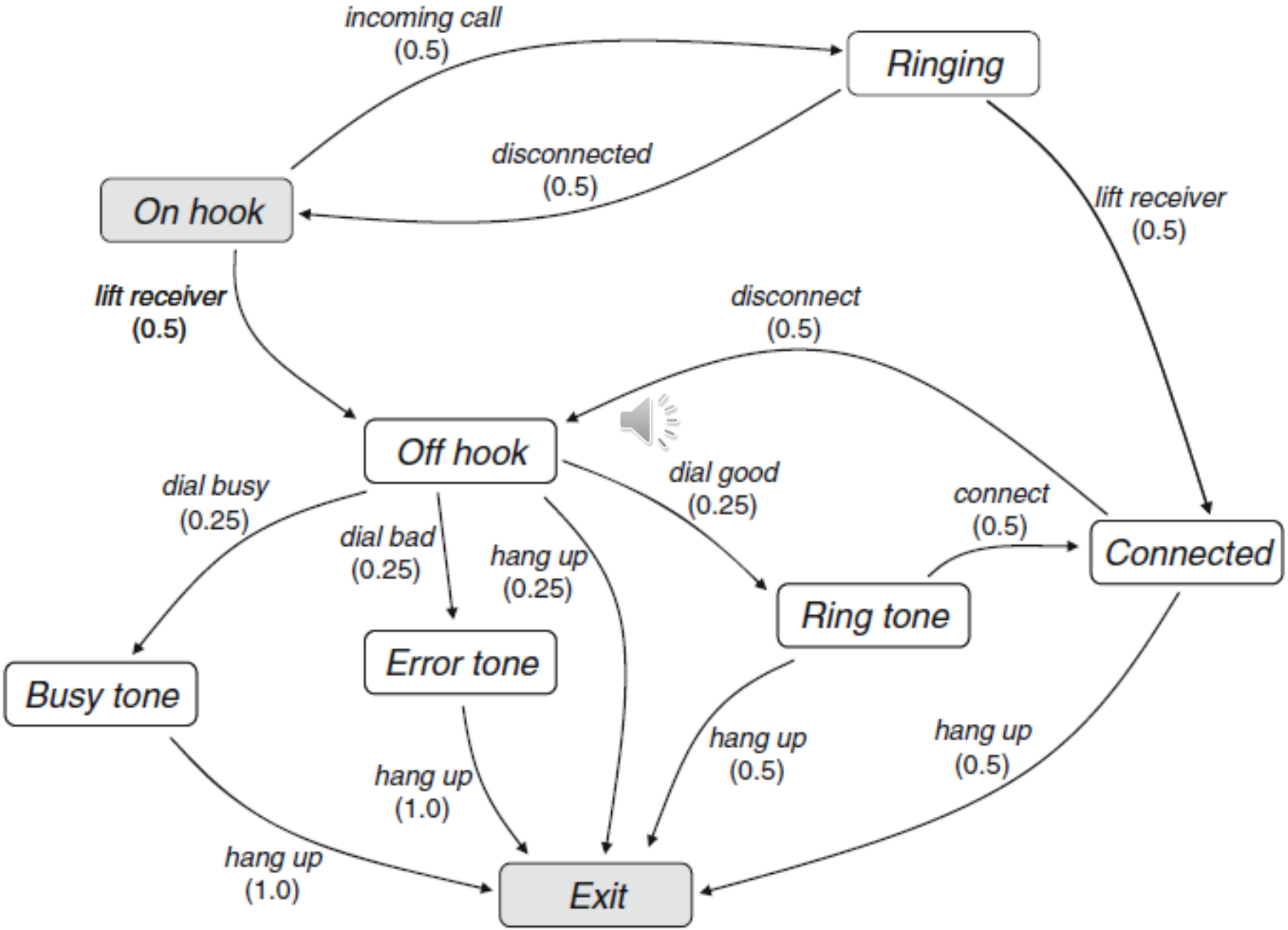
MBST process overview



- A functional system specification is derived from the system requirements by applying the sequence-based specification technique (not explained here).
- Then, the system usage is modelled and a Markov chain usage model is derived.
- Test case generation, execution, and evaluation are automated using appropriate tools.

Usage modelling for MBST

- A usage model describes how the system under test is expected to be used.
- Usage models are finite state machines with transition probabilities.
- The transitions describe the possible system triggers by external stimuli and the expected system responses.
- The probabilities express the frequency of system stimuli at certain states and form the probability distribution of the usage model.
- Several usage profiles might be annotated for one test object, e.g., for different user groups.
- Usage models have one start state and one exit state, which correspond to the beginning and end of a system use.
- Any path through the model from the start state to the exit state represents a valid system use.


incoming call
(0.5)
Ringing
disconnected
(0.5)
On hook
lift receiver
(0.5)
lift receiver
(0.5)
disconnect
(0.5)
Off hook
dial busy
(0.25)
dial bad
(0.25)
hang up
(0.25)
dial good
(0.25)
connect
(0.5)
Connected
Ring tone
Busy tone
Error tone
hang up
(0.5)
hang up
(0.5)
hang up
(1.0)
hang up
(1.0)
Exit

Test case generation for MBST

- A test case is a path through the usage model.
- The transitions of the usage model are annotated with test scripts that drive a test runner.
- To generate a specific test case, the scripts of the transitions lying on a path through the usage model will be appended one after another and saved as a single test case script.
- During a test case, the test runner runs each test script sequentially.

Test execution for MBST

- Automated test execution needs a test runner that executes the generated test cases.
- A test case is a sequence of test scripts (each representing a test step) collected from the usage model transitions during test generation.
- The transition from one usage model state to another is performed by invoking the corresponding test script.
- Test scripts are written in an executable notation.

Test evaluation for MBST

- The test scripts in the test cases include the required input for the test object and the expected output from the test object.
- A test case is considered to have failed if at least one test script does not observe the specified output.
- For example, consider the telephone again.
- One possible test script would be to try to lift the receiver.
- The test step fails if the script does not observe a free line signal.

Test analysis for MBST

- Based on the test results, the reliability of the test object is estimated by using a particular reliability model, the Miller model.
- Reliability increases with successful executions and decreases after failures. 
- The goal is to achieve a reliability of close to 1.
- The reliability of the test object is the weighted sum of all transition reliabilities.
- It is also feasible to calculate the so-called optimum reliability, which is the highest reliability estimated that can be achieved by applying the current set of test cases, assuming no failures.

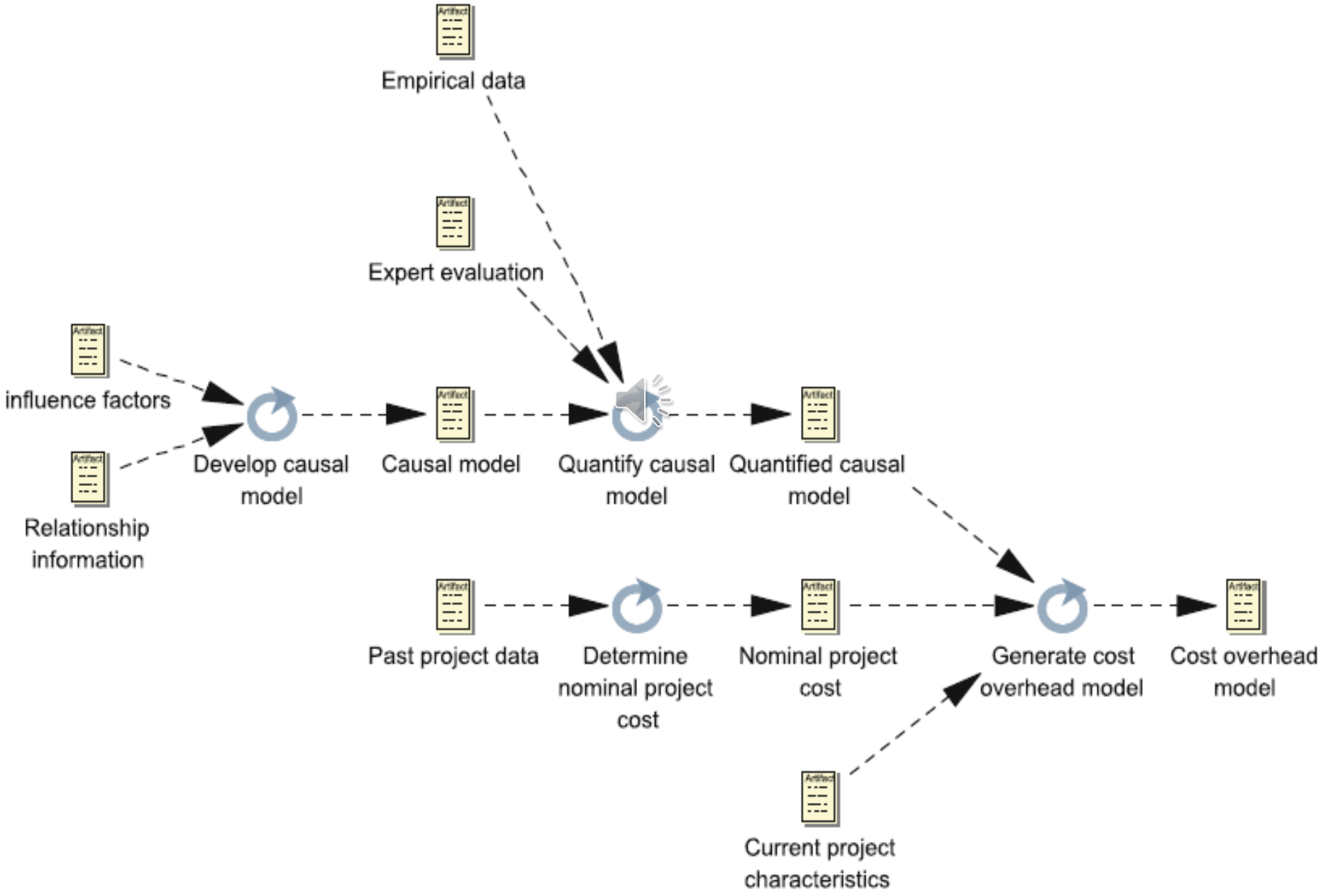
Process for Hybrid Cost Estimation

- Cost Estimation, Benchmarking and Risk Analysis (CoBRA) method
- The CoBRA® method for cost estimation is a method for the systematic utilization of expert and empirical knowledge to estimate the costs for an upcoming project.
- Based on a combination of measurement data and expert judgment, it predicts the probable costs for projects of the same kind.
- Important - CoBRA® focuses on a very small fraction of the complete software product development lifecycle (cost estimation at project start) and gives very detailed instructions on what to do and how to do it.

Steps of CoBRA

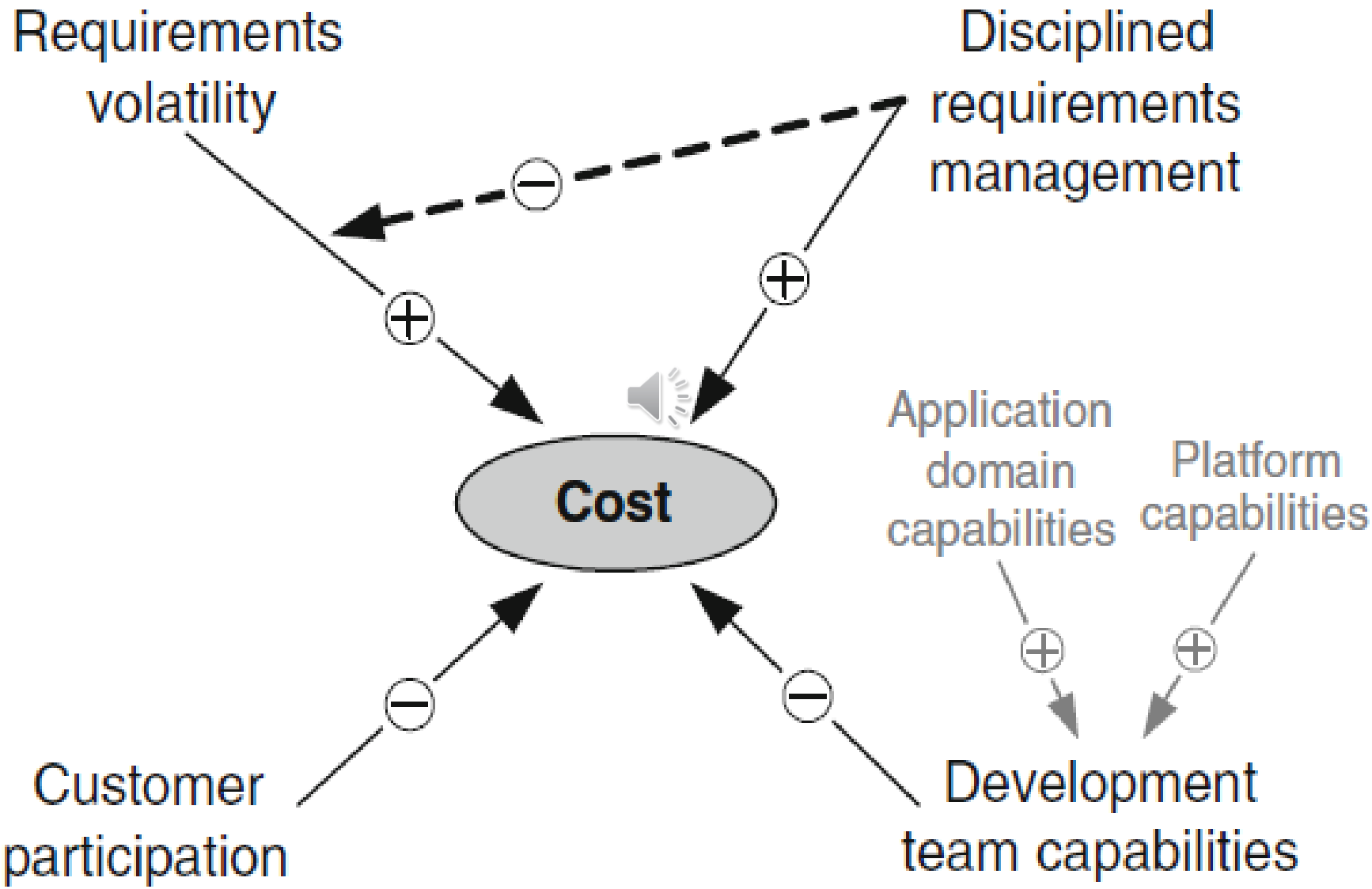
- The method involves four major steps:

- Develop causal model,
- Quantify causal model,
- Determine nominal project costs, and
- Generate cost overhead model.


Empirical data
Expert evaluation
influence factors
Relationship information
Develop causal model
Causal model
Quantify causal model
Quantified causal model
Past project data
Determine nominal project cost
Nominal project cost
Generate cost overhead model
Cost overhead model
Current project characteristics

Develop a Causal Model

- The causal model contains factors affecting the costs of
- projects within a certain context.
- The causal model is typically obtained through expert knowledge acquisition (e.g., involving experienced project managers).
- An example is presented in the next figure.
- The arrows indicate direct and indirect relationships.
- A sign (“+” or “–“) indicates the way a cost factor contributes to the overall project costs.
- The “+” and “-” represent a positive and negative relationship, respectively; that is, if the factor increases, the project costs will also increase(“+”) or decrease (“-”).


Requirements volatility
Disciplined requirements management
Cost
Application domain capabilities
Platform capabilities
Customer participation
Development team capabilities

- If requirements volatility increases, costs will also increase.
- One arrow pointing to another one indicates an interaction effect.
- For example, an interaction exists between disciplined requirements management and requirements volatility.
- In this case, increased disciplined requirement management compensates for the negative influence of volatile requirements on software costs.

Quantify Causal Model

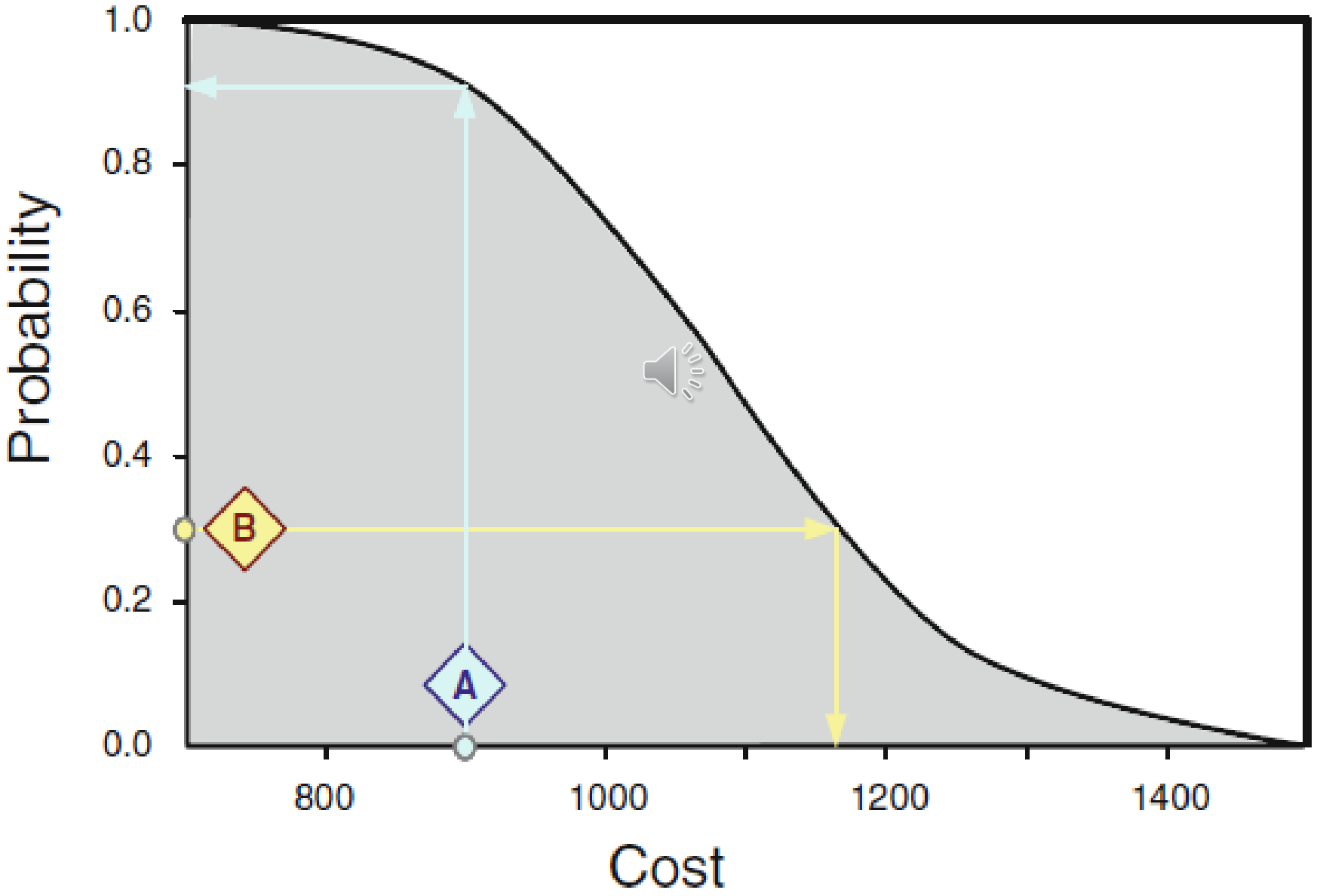
- The direct and indirect (through other factors) impact on costs for each factor is quantified using expert evaluation and/or empirical data.
- The influence is measured as a relative percentage increase of the costs above a nominal project (i.e., a “perfect” project where everything runs optimally).
- For each factor, experts are asked to give the increase in costs when the considered factor has the worst possible value (extreme case) and all other factors have their nominal values (best case).
- In order to capture the uncertainty of evaluations, experts are asked to give three values: the maximal, minimal, and most likely cost overhead for each factor.

Determine nominal project cost.

- The nominal project costs, i.e., the costs of a “perfect” project where everything runs optimally, are based on data from past projects that are similar with respect to certain characteristics (e.g., development type, lifecycle type) that are not part of the causal model.
- These characteristics define the context of the project. Past project data is used to determine the relationship between cost overhead and (nominal) costs.
- Since it is a simple bivariate dependency, it does not require much measurement data. In principle, merely project size and effort are required.
- The size measure should reflect the overall project volume including all produced artefacts.
- Common examples include lines of code or function points. Past project information on identified cost factors is usually elicited from experts.

Generate cost overhead model

- Based on the quantified causal model, past project data, and current project characteristics, a cost overhead model is generated for the current project using a simulation algorithm (e.g., Monte Carlo or Latin Hypercube).
- The probability distribution obtained could be used further to support various project management activities, such as cost estimation, evaluation of cost related project risks, or benchmarking.
- An example with two usage scenarios using the inverse cumulative cost distribution:
- computing the probability of exceeding given project costs (scenario A) and calculating the project costs for a given probability level (scenario B).


1.0
0.8
0.6
0.4
0.2
0.0
Probability
B
A
800
1000
1200
1400
Cost

Example

- Let us assume (scenario A) that the available budget for a project is 900,000 euros and that this project's costs are characterized by the distribution in the figure.

- There is roughly a 90% probability that the project will overrun this budget.

- If this probability represents an unacceptable risk in a particular context, the project budget might not be approved.

Example

- Let us consider (scenario B) that a project manager wants to minimize the risks of overrunning the budget.
- In other words, the costs of a software project should be planned so that there is minimal risk of exceeding them.
- If a project manager sets the maximal tolerable risk of exceeding the budget to 30%, then the planned budget for the project should not be lower than 1,170,000 (ish) euros.
- Further reading - COBRA: A Hybrid Method for Software Cost Estimation, Benchmarking, and Risk Assessment, Briand et al.

Extreme Programming (XP)

- Extreme Programming (XP) is an agile approach to software development with a strong focus on producing code, whereas all other activities are only executed if they provide direct benefit for the customer.
- Requirements are represented by using Story Points: short few lines of statements on what the customer wants.
- Follows a set of principles and best practices.
- In many cases, when organizations claim to do “agile development” or XP, they merely apply one or two practices, but follow a chaotic process otherwise.

- Self study – Find, read and master yourself on XP!

Process Standards

- There are widely used software process standards for reference.
- ISO/IEC 12207:1995 and ISO/IEC 12207:2008 are the basis for many other, derived standards, such as ISO/IEC 15504 (SPICE) or the European Space Agency's ECSS standards.
- IEC 61508 [23] is commonly used for the development of safety-critical systems such as automotive or aerospace (embedded) systems.
- ISO – International Organisation for Standardising
- IEC – International Electrotechnical Commission

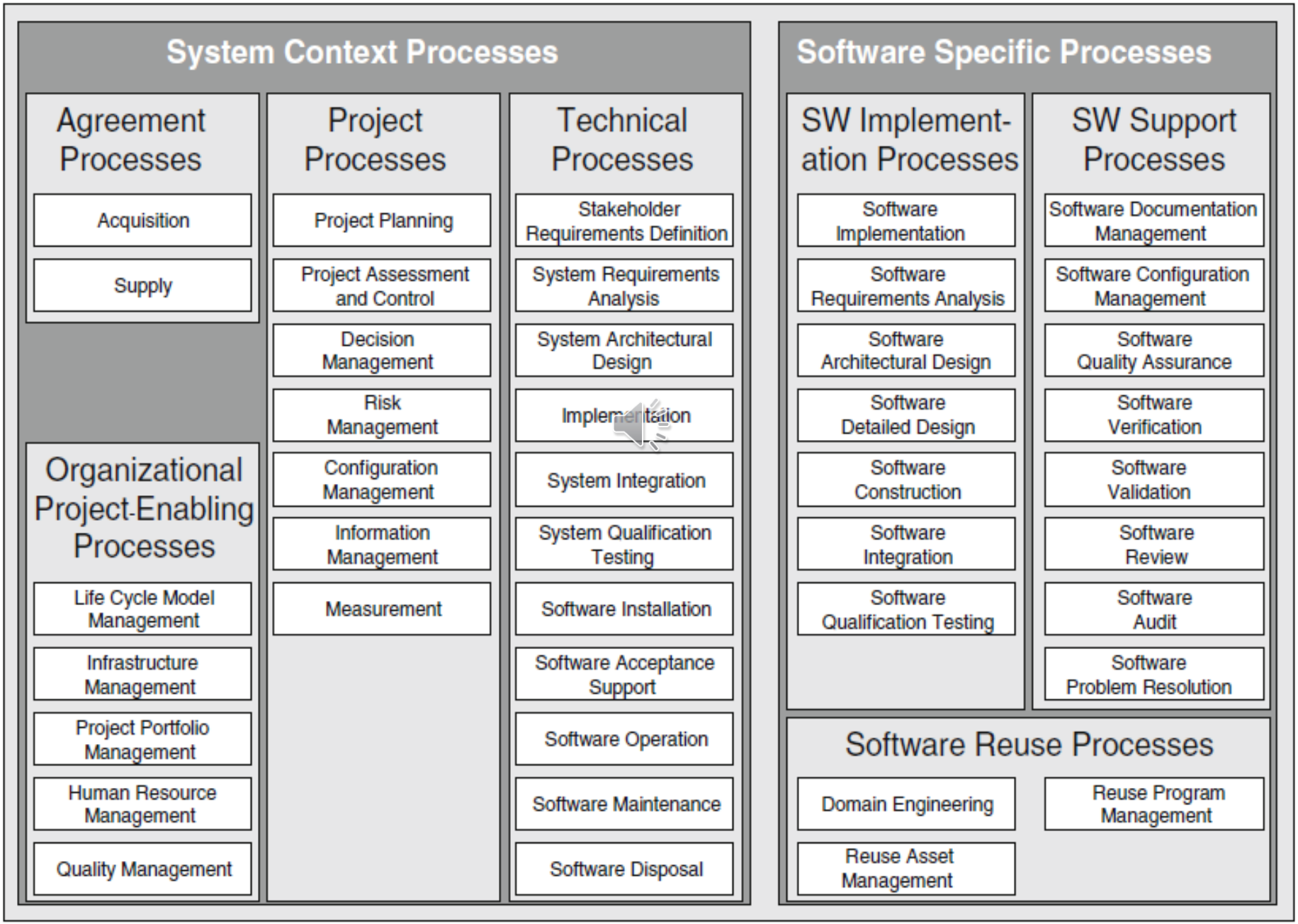
ISO/IEC 12207:2008

- The international standard ISO/IEC 12207:2008 [20] is a process standard that establishes a common framework for software lifecycle processes, with a well defined terminology, which can be referenced by the software industry.
- The standard covers the software lifecycle from collecting the first ideas and concepts via implementing the system to decommissioning obsolete systems.
- Many other standards, such as CMMI or ISO/IEC 15504 (SPICE), are derived directly or indirectly from ISO/IEC 12207 (version 95 or 08)

- The standard groups the activities executed during the software lifecycle into seven process groups.
- Each of the 44 processes described within the process groups contains a purpose statement and a list of expected outcomes.
- Furthermore, activities and tasks that are necessary to produce the expected outcomes are described.

- The figure shows an overview of the standard.

- In contrast to its predecessor (ISO/IEC 12207:1995), ISO/IEC 12207:2008 distinguishes processes for standalone software products or services (System Context Processes, left-hand part of the figure) and processes for software products or services that are part of a superior system (Software Specific Processes, right-hand part of the figure).


System Context Processes
Agreement Processes
Acquisition
Supply
Organizational Project-Enabling Processes
Life Cycle Model Management
Infrastructure Management
Project Portfolio Management
Human Resource Management
Quality Management
Project Processes
Project Planning
Project Assessment and Control
Decision Management
Risk Management
Configuration Management
Information Management
Measurement
Technical Processes
Stakeholder Requirements Definition
System Requirements Analysis
System Architectural Design
Implementation
System Integration
System Qualification Testing
Software Installation
Software Acceptance Support
Software Operation
Software Maintenance
Software Disposal
Software Specific Processes
SW Implement-ation Processes
Software Implementation
Software Requirements Analysis
Software Architectural Design
Software Detailed Design
Software Construction
Software Integration
Software Qualification Testing
SW Support Processes
Software Documentation Management
Software Configuration Management
Software Quality Assurance
Software Verification
Software Validation
Software Review
Software Audit
Software Problem Resolution
Software Reuse Processes
Domain Engineering
Reuse Asset Management
Reuse Program Management

System Context Processes

- The process group Agreement Processes contains the processes necessary to reach an agreement between two organizations.
- Organizational Project-Enabling Processes are processes that facilitate and support project execution on a strategic level.
- Project Processes deal with “classic” project management tasks, i.e., planning and controlling, decision management, risk management, configuration and information management, and measurement.
- Product development, including (system) requirements elicitation, system architecture, implementation, integration, testing, installation, operation, maintenance, and disposal, is described by the Technical Processes.

Software Specific Processes

- The Software Implementation Processes describe the activities necessary to create a specific system element that is implemented in software.
- This includes software requirements analysis, software architecture and design, implementation, integration, and testing.
- Support Processes contain software-specific processes with respect to documentation, configuration management, quality assurance, verification and validation, reviews and audits, and problem resolution.
- Finally, Software Reuse Processes define processes for systematically reusing system components.

- ISO/IEC 12207 is sometimes called “the mother of all standards,” because most specialized standards are based directly or indirectly on it.

- Beware: Many derived standards still reference ISO/IEC 12207:1995, the predecessor of ISO/IEC 12207:2008!